

# TokenPack Demo Knowledge Base

Prepared for Retrieval and Knapsack Testing

## 1 Project Overview

TokenPack is a prototype retrieval system that treats a language model context window as a limited token budget. A long document is divided into semantic chunks. Each chunk receives a relevance value according to how similar it is to the user query. Each chunk also has a token weight. The goal is to select the most useful chunks without exceeding the context budget.

The main idea is based on the 0/1 Knapsack Problem. In this mapping, every chunk is an item, the token count is the item weight, the semantic relevance score is the item value, and the available context window is the knapsack capacity. This formulation is useful because language models cannot always read every available document at once.

TokenPack is designed as a Python package and command-line tool. It can ingest text or PDF files, create chunks, compute embeddings, compare retrieval strategies, export selected context, and optionally send the selected context to a local language model.

## 2 Context Window Budget

A context window is the maximum amount of text that a language model can receive in a single request. TokenPack separates the total context window into two parts. One part is reserved for retrieved chunks, and the other part is reserved for the model's answer. For example, if the total budget is 50,000 tokens and 4,000 tokens are reserved for the answer, then 46,000 tokens remain for selected chunks.

Budget awareness is important because a retrieval method may find relevant chunks but still fail if the selected text is too long. A top-k retriever can select the highest ranked chunks without checking their total token cost. In contrast, a budget-aware method must ensure that the final selected context never exceeds the effective budget.

In TokenPack, over-budget selection is treated as invalid for production use. A method can be fast and relevant, but if it violates the token limit, it cannot be safely used as the final context packer.

## 3 Semantic Chunking

Semantic chunking is the process of dividing a document into meaningful pieces. TokenPack supports paragraph-group chunking and semantic-threshold chunking. Paragraph-group chunking groups neighboring paragraphs until a target token range is reached. This approach keeps local context together and avoids cutting every fixed number of words.

Semantic-threshold chunking compares neighboring text blocks using embedding similarity. If the similarity between two neighboring blocks drops below a threshold, TokenPack treats that location as a topic change and starts a new chunk. This helps keep unrelated topics in separate chunks.

Chunking quality affects retrieval quality. If a chunk is too small, it may lose the surrounding explanation. If a chunk is too large, it may waste the context budget. TokenPack therefore stores chunk metadata such as source file, page number, paragraph order, and token count so that selected chunks can later be restored to their original order.

## 4 Retrieval Strategies

TokenPack compares several retrieval and selection strategies. Top-k retrieval selects a fixed number of chunks by similarity score. Budget-top-k follows the similarity ranking but skips chunks that would exceed the remaining budget. MMR, or Maximal Marginal Relevance, also considers redundancy and tries to avoid selecting chunks that are too similar to each other.

The knapsack strategy solves a 0/1 optimization problem. It tries to maximize total relevance value while keeping the selected token count under the budget. Dynamic programming can compute the exact optimal solution when the number of candidate chunks and the budget are manageable.

Greedy strategies are faster but not always optimal. Greedy by value selects high-value chunks first. Greedy by value density selects chunks with high value per token. Simulated annealing is a metaheuristic that explores neighboring selections and can sometimes improve over simple greedy choices, although it usually takes more time.

## 5 Evaluation Procedure

TokenPack evaluation compares strategies under the same budget. Important metrics include evidence recall, evidence precision, selected token count, budget utilization, value density, redundancy score, and latency. Evidence recall measures whether the selected context includes the known evidence chunks needed to answer a query.

For algorithm analysis, TokenPack also runs repeated synthetic knapsack experiments. The same randomly generated instances are solved by exact dynamic programming, greedy methods, simulated annealing, and random feasible selection. This makes it possible to measure runtime, approximation gap, timeout rate, and confidence intervals.

The expected pattern is that dynamic programming gives the optimal answer for small and medium instances, but becomes expensive when the state space grows. Greedy density is usually much faster and often close to the optimum. Value-only greedy can be poor because it ignores token weight.

## 6 Worked Example

Assume that a user asks why top-k retrieval can be unsafe under a strict context window. The document contains three candidate chunks. Chunk A explains top-k retrieval and has high relevance, but it has 700 tokens. Chunk B explains token budgeting and has medium relevance with 250 tokens. Chunk C is a short definition with 100 tokens and low relevance. If the remaining context budget is only 350 tokens, a naive top-k method may try to select Chunk A and fail the budget constraint. A budget-aware method can instead choose Chunk B and Chunk C if their combined value is useful and their total token count fits.

This example shows why TokenPack treats token count as a first-class optimization variable. The best chunk is not always the best final decision if it is too large for the remaining context. A context packer must consider both semantic value and token cost at the same time.

The same logic applies to long PDF files. A large paragraph may contain useful information, but including it can prevent several smaller evidence chunks from fitting into the prompt. Knapsack selection is useful because it evaluates combinations of chunks rather than only looking at one chunk at a time.

## 7 Gold Review Facts

This demo document includes simple facts for manual gold evidence review. Fact one: TokenPack maps chunks to knapsack items. Fact two: chunk token count is the weight. Fact three: semantic relevance is the value. Fact four: the context window is the capacity. Fact five: top-k can be unsafe because it may ignore total token cost. Fact six: semantic-threshold chunking starts a new chunk when neighboring block similarity drops below a threshold. Fact seven: evidence recall checks whether selected chunks contain the known supporting evidence. Fact eight: local generation can use Ollama without an OpenAI API key.

These facts are intentionally direct. A reviewer should be able to open the review packet, compare each answer with the quoted evidence text, and decide whether the gold record is valid in less than a minute per question.

## 8 Local Language Model Usage

TokenPack can run without an external API key. For local generation, it can use an Ollama model installed on the same computer. The retrieval benchmark itself does not require a language model, because it evaluates whether the correct evidence chunks are selected under the token budget.

Local generation is useful as a demo layer. After TokenPack selects chunks, the selected context can be exported and given to a local model with a question. The model should answer only from the provided context. This separation is intentional: retrieval quality can be measured offline, while answer generation can be tested later with different local models.

For the first project version, the most important success criterion is not whether the model writes a beautiful final answer. The main success criterion is whether TokenPack can select valuable evidence chunks while respecting the context budget.